Lecture Notes: Sampling Algorithms, Markov Random Fields, and Markov Chain Monte Carlo

Based on the lectures of Peter Orbanz

Contents

1 Sampling Algorithms

1.1 Motivation and Overview

Definition 1.1 (Sampling Algorithm)**.** A **sampling algorithm** takes a distribution P as input and outputs random values $X_1, \ldots, X_n$ whose marginal distribution is P. Ideally, these draws are independent.

Distributions that are hard to work with analytically may nonetheless be relatively easy to sample from. Given samples, we can:

- **Compute expectations.** If $X_1, \ldots, X_n$ are independent,

$$\mathbb{E}_P\big[f(X)\big] \;\approx\; \frac{1}{n}\sum_{i=1}^{n} f(X_i) \tag{1}$$

  by the law of large numbers (for a given function f).
- **Approximate the distribution** by using the samples as input for density estimation.

Remark 1.2 (Inference in Bayesian Models). Suppose we work with a Bayesian model whose posterior $\hat{Q}_n := \mathcal{L}(\Theta \mid X_{1:n})$ cannot be computed analytically. It may still be possible to *sample* from $\hat{Q}_n$, thereby obtaining samples $\Theta_1, \Theta_2, \ldots$ distributed according to $\hat{Q}_n$. This reduces posterior estimation to a density estimation problem.

1.2 Posterior Expectations and Predictive Distributions

If we are only interested in some statistic of the posterior of the form $\mathbb{E}_{\hat{Q}_n}[f(\Theta)]$ (e.g. the posterior mean), we can approximate by

$$\mathbb{E}_{\hat{Q}_n}\big[f(\Theta)\big] \;\approx\; \frac{1}{m}\sum_{i=1}^{m} f(\Theta_i). \tag{2}$$

Example 1.3 (Posterior Predictive Distribution)**.** The **posterior predictive distribution** is our best guess of what the next data point x_{n+1} looks like, given the posterior under previous observations. In terms of densities:

$$p(x_{n+1} \mid x_{1:n}) := \int_{\mathcal{T}} p(x_{n+1} \mid \theta)\, \hat{Q}_n(d\theta \mid X_{1:n} = x_{1:n}). \tag{3}$$

This is a posterior expectation (Definition 1.1, (2)) and can be approximated as:

$$p(x_{n+1} \mid x_{1:n}) = \mathbb{E}_{\hat{Q}_n}\big[p(x_{n+1} \mid \Theta)\big] \;\approx\; \frac{1}{m}\sum_{i=1}^{m} p(x_{n+1} \mid \Theta_i). \tag{4}$$

1.3 Rejection Sampling

1.3.1 The Key Idea

Consider a probability density p on the interval $[a, b]$. Let A denote the area under the curve of p. Suppose we can define a uniform distribution U_A on A. If we generate $(X_1, Y_1), (X_2, Y_2), \ldots \overset{\text{iid}}{\sim} U_A$, then $X_1, X_2, \ldots \overset{\text{iid}}{\sim} p$.

Algorithm 1.4 (Rejection Sampling on the Interval)**.** Given a density p on $[a, b]$ bounded above by a constant $c > 0$:

1. Generate (X_i, Y_i) uniformly on the enclosing box $B = [a, b] \times [0, c]$, by independently sampling $X_i \sim \text{Uniform}[a, b]$ and $Y_i \sim \text{Uniform}[0, c]$.
2. If $Y_i \leq p(X_i)$, keep the sample; otherwise, discard (reject) it.

The retained samples are uniformly distributed on A and hence $X_1, X_2, \ldots \sim p$.

1.3.2 Distributions Known up to Scaling

Remark 1.5 (Sufficiency of Unnormalized Densities). For sampling, it suffices to know a distribution only up to normalization. That is:

$$p(x) = \frac{1}{\tilde{Z}}\,\tilde{p}(x), \tag{5}$$

and we can compute $\tilde{p}(x)$ for any given x, but we do not know $\tilde{Z}$. This is because the rejection criterion $Y_i < \tilde{p}(X_i)$ depends only on the shape of p, not on the normalizing constant.

There are important cases where we can evaluate only the unnormalized density $\tilde{p}$.

Example 1.6 (Simple Posterior)**.** For an arbitrary posterior computed with Bayes' theorem:

$$\Pi(\theta \mid x_{1:n}) = \frac{\prod_{i=1}^{n} p(x_i \mid \theta)\, q(\theta)}{\tilde{Z}}, \qquad \tilde{Z} = \int_{\mathcal{T}} \prod_{i=1}^{n} p(x_i \mid \theta)\, q(\theta)\, d\theta.$$

Provided that we can compute the numerator, we can sample without computing the normalization integral $\tilde{Z}$.

Example 1.7 (Bayesian Mixture Model)**.** The posterior of a Bayesian mixture model (BMM) is, up to normalization:

$$\hat{q}_n(c_{1:K}, \theta_{1:K} \mid x_{1:n}) \;\propto\; \prod_{i=1}^{n}\Big(\sum_{k=1}^{K} c_k\, p(x_i \mid \theta_k)\Big) \prod_{k=1}^{K} q(\theta_k) \cdot q_{\mathrm{Dir}}(c_{1:K}). \tag{6}$$

Computing $\tilde{q}_n$ as a formula in terms of unknowns is difficult (the product–sum blows up into K^n terms), but *evaluating* it for specific values of (c, x, θ) is easy.

Example 1.8 (Markov Random Field)**.** In a Markov random field (see Section 2), the normalization function is the real problem. For the Ising model (Definition 2.8):

$$Z(\beta) = \sum_{\theta_{1:n} \in \{0,1\}^n} \exp\Big(\sum_{(i,j) \text{ is an edge}} \beta\, \mathbb{I}\{\theta_i = \theta_j\}\Big),$$

which is a sum over 2^n terms. On the other hand, evaluating $\tilde{p}(\theta_{1:n}) = \exp\big(\sum_{(i,j)} \beta\, \mathbb{I}\{\theta_i = \theta_j\}\big)$ for a given configuration is straightforward.

1.3.3 Rejection Sampling on $\mathbb{R}^d$

On $\mathbb{R}^d$ there is no uniform distribution on all of $\mathbb{R}^d$, so we replace the enclosing box with a **proposal density** r.

Algorithm 1.9 (Rejection Sampling on $\mathbb{R}^d$)**.** Choose a distribution r from which we know how to sample. Scale $\tilde{p}$ such that $\tilde{p}(x) < r(x)$ everywhere. For $i = 1, 2, \ldots$:

1. Sample $X_i \sim r$.
2. Sample $Y_i \mid X_i \sim \mathrm{Uniform}[0, r(X_i)]$.
3. If $Y_i < \tilde{p}(X_i)$, keep X_i; else, discard X_i and repeat.

The retained samples are distributed according to p.

Proposition 1.10 (Properties of Rejection Sampling)**.**

1. ***Independence.*** *If we draw proposal samples i.i.d. from r, the accepted samples are i.i.d. Hence $\frac{1}{m}\sum_{i \le m} f(X_i)$ is an unbiased estimate of $\mathbb{E}_p[f(X)]$.*
2. ***Efficiency.*** *The fraction of accepted samples equals the ratio $|A|/|B|$ of the areas under $\tilde{p}$ and r. If r is not a reasonably close approximation of p, most proposals are rejected.*

Remark 1.11 (High-Dimensional Challenges). Sampling is typically used for distributions of multiple, dependent random variables. High-dimensional distributions with dependence often concentrate on many small areas strewn across the sample space, with regions of effectively zero probability in between. In such settings, the acceptance ratio of rejection sampling can be as low as 10^{-6} or much worse. Even in moderate dimensions, this is not the exception but the rule.

1.4 Importance Sampling

There is a simple way to improve on rejection sampling (Algorithm 1.9) if we are specifically interested in approximating an expectation $\mathbb{E}_p[f(X)]$.

1.4.1 Simple Case: p Evaluable

Let p be the target and q a proposal density. Rewrite:

$$\mathbb{E}_p\big[f(X)\big] = \int f(x)\,p(x)\,dx = \int f(x)\,\frac{p(x)}{q(x)}\,q(x)\,dx = \mathbb{E}_q\left[\frac{f(X)\,p(X)}{q(X)}\right]. \tag{7}$$

Algorithm 1.12 (Importance Sampling)**.** Draw $X_1, X_2, \ldots \overset{\text{iid}}{\sim} q$. Do not discard any samples. Approximate the expectation of f as:

$$\mathbb{E}_p\big[f(X)\big] \;\approx\; \frac{1}{m}\sum_{i\le m} f(X_i)\,\frac{p(X_i)}{q(X_i)}. \tag{8}$$

The coefficients $w(X_i) := p(X_i)/q(X_i)$ are called **importance weights**. There are no rejections, but some samples may carry small weights.

1.4.2 General Case: p Known up to Scaling

Now assume we can only evaluate p and q up to scaling:

$$p = \frac{1}{Z_p}\,\tilde{p} \qquad \text{and} \qquad q = \frac{1}{Z_q}\,\tilde{q},$$

where Z_p (and possibly Z_q) are unknown constants.

Proposition 1.13 (Self-Normalized Importance Sampling)**.** *We can estimate the ratio Z_p/Z_q using samples $X_1, \ldots, X_m \overset{iid}{\sim} q$:*

$$\frac{Z_p}{Z_q} = \mathbb{E}_q\left[\frac{\tilde{p}(X)}{\tilde{q}(X)}\right] \;\approx\; \frac{1}{m}\sum_{i\le m}\frac{\tilde{p}(X_i)}{\tilde{q}(X_i)}. \tag{9}$$

The self-normalized estimator of the expectation of f is:

$$\mathbb{E}_p\big[f(X)\big] \;\approx\; \frac{\sum_{i=1}^m f(X_i)\,\frac{\tilde{p}(X_i)}{\tilde{q}(X_i)}}{\sum_{j=1}^m \frac{\tilde{p}(X_j)}{\tilde{q}(X_j)}}. \tag{10}$$

Algorithm 1.14 (Importance Sampling — p Known up to Scaling)**.** Draw $X_1, X_2, \ldots \overset{\text{iid}}{\sim} q$, and approximate the expectation as in (10).

2 Markov Random Fields

2.1 Random Fields

Definition 2.1 (Neighborhood Graph)**.** A **neighborhood graph** is a weighted, undirected graph

$$N = (V_N,\, W_N),$$

where V_N is the vertex set and W_N is the set of edge weights $w_{ij} \in \mathbb{R}$. Since N is undirected, $w_{ij} = w_{ji}$. An edge weight $w_{ij} = 0$ means "no edge between v_i and v_j".

Definition 2.2 (Random Field)**.** With each vertex v_i in a neighborhood graph (Definition 2.1), we associate a random variable Θ_i. The joint distribution of these variables is called a **random field**.

Definition 2.3 (Neighborhood and Markov Blanket)**.** The **neighborhood** of vertex v_i is the set

$$\partial(i) := \{\, j \mid w_{ij} \neq 0 \,\}. \tag{11}$$

The set $\{\Theta_j : j \in \partial(i)\}$ of random variables associated with the neighborhood is the **Markov blanket** of Θ_i.

2.2 Markov Random Fields

Definition 2.4 (Markov Property and MRF)**.** A random field (Definition 2.2) has the **Markov property** if

$$P(\theta_i \mid \theta_j,\, j \neq i) = P(\theta_i \mid \theta_j,\, j \in \partial(i)). \tag{12}$$

That is, each Θ_i is conditionally independent of the remaining field given its Markov blanket (Definition 2.3). A random field satisfying the Markov property is called a **Markov random field (MRF)**.

Definition 2.5 (Energy Function)**.** Any strictly positive probability or density p can be written in the form

$$p(x) = \frac{1}{Z}\exp\big(-H(x)\big) \qquad \text{where} \quad H : \mathcal{X} \to \mathbb{R}_+ \quad \text{and} \quad Z := \int e^{-H(x)}\,dx. \tag{13}$$

The function H is called an **energy function** (or potential, or cost function), and Z is the normalization constant (partition function). In particular, we can write an MRF density for random variables $\Theta_1, \ldots, \Theta_n$ as

$$p(\theta_1, \ldots, \theta_n) = \frac{1}{Z}\exp\big(-H(\theta_1, \ldots, \theta_n)\big).$$

2.3 The Potts Model

Definition 2.6 (Potts Model)**.** Let N be a neighborhood graph (Definition 2.1) with weights w_{ij}, and let $\beta > 0$. The Markov random field

$$p(\theta_1, \ldots, \theta_n) := \frac{1}{Z(\beta)}\exp\Big(\beta \sum_{i,j} w_{ij}\,\mathbb{I}\{\theta_i = \theta_j\}\Big) \tag{14}$$

is called a **Potts model**.

Remark 2.7 (Effect of Edge Weights). The energy in the Potts model (Definition 2.6) is additive over pairs. Positive weights encourage smoothness: $w_{ij} > 0$ and $\theta_i = \theta_j$ increases probability; $w_{ij} < 0$ and $\theta_i = \theta_j$ decreases probability; $w_{ij} = 0$ means no interaction between θ_i and θ_j.

2.4 The Ising Model

Definition 2.8 (Ising Model)**.** If $\theta_i \in \{-1, +1\}$ and $w_{ij} \in \{0, 1\}$ in the Potts model (Definition 2.6), we obtain

$$p(\theta_1, \ldots, \theta_n) = \frac{1}{Z(\beta)}\exp\Big(\sum_{(i,j)\text{ is an edge}} \beta\,\mathbb{I}\{\theta_i = \theta_j\}\Big). \tag{15}$$

If N is a d-dimensional grid, this model is called the **Ising model**. This is the simplest non-trivial Potts model, but many of its mathematical properties remain unsolved.

2.5 MRFs as Smoothness Priors

Definition 2.9 (Spatial Model with MRF Prior)**.** Consider a spatial problem with observations X_i, where each i is a location on a grid. We model each X_i by a distribution $\mathcal{L}(X \mid \Theta_i)$, so each location i has its own parameter variable Θ_i. We place an MRF (Definition 2.4) as a prior distribution on $(\Theta_1, \ldots, \Theta_n)$.

Remark 2.10 (Spatial Smoothing). If we define the joint distribution of $(\Theta_1, \ldots, \Theta_n)$ as an MRF on the grid graph with positive weights, the MRF will encourage the model to explain neighboring observations X_i and X_j by the same parameter value. This produces **spatial smoothing**, analogous to the emission structure in an HMM.

2.6 Bayesian Mixture Models and Image Segmentation

Definition 2.11 (Bayesian Mixture Model)**.** A model of the form

$$\pi(x) = \sum_{k=1}^{K} C_k \, p(x \mid \Theta_k) \tag{16}$$

is called a **Bayesian mixture model (BMM)** if $p(x \mid \theta)$ is an exponential family model and:

- $\Theta_1, \ldots, \Theta_K \overset{\text{iid}}{\sim} q$, where q is a prior on Θ;
- $(C_1, \ldots, C_K)$ is sampled from a K-dimensional Dirichlet distribution.

The posterior distribution of a BMM under observations $x_1, \ldots, x_n$ is (up to normalization):

$$\Pi(c_{1:K}, \theta_{1:K} \mid x_{1:n}) \;\propto\; \prod_{i=1}^{n} \Big(\sum_{k=1}^{K} c_k \, p(x_i \mid \theta_k) \Big) \prod_{k=1}^{K} q(\theta_k) \cdot q_{\text{Dir}}(c_{1:K}). \tag{17}$$

Example 2.12 (Segmentation of Noisy Images)**.** In the spatial setting, the BMM can be used for image segmentation by indexing the parameter of each X_i separately as θ_i. For K mixture components, $\theta_{1:n}$ contains only K different values. The joint BMM prior on $\theta_{1:n}$ is $q_{\text{BMM}}(\theta_{1:n}) = \prod_{i=1}^{n} q(\theta_i)$. We multiply this with an MRF prior (Definition 2.4):

$$q_{\text{MRF}}(\theta_{1:n}) = \frac{1}{Z(\beta)} \exp\Big(\beta \sum_{w_{ij} \neq 0} \mathbb{I}\{\theta_i = \theta_j\} \Big) \tag{18}$$

to encourage spatial smoothness of the segmentation.

2.7 Sampling and Inference in MRFs

MRFs pose two main computational problems:

1. **Sampling.** Generate samples from the joint distribution of $(\Theta_1, \ldots, \Theta_n)$.
2. **Inference.** If the MRF is used as a prior, compute or approximate the posterior distribution.

Remark 2.13 (Intractability of MRF Distributions). MRF distributions on grids are not analytically tractable. The only known exception is the Ising model (Definition 2.8) in one dimension. Both sampling and inference are based on Markov chain sampling algorithms (Section 3).

3 Markov Chain Monte Carlo

3.1 Motivation

Remark 3.1 (Limitation of Rejection Sampling). In rejection sampling (Algorithm 1.9), proposals are i.i.d.: once we find a region where p concentrates, we "forget" about it for the next sample. For distributions that concentrate on narrow regions, we would like each sample to depend on the location of the previous sample, so that we can continue drawing within a high-probability region.

3.2 The MCMC Idea

Definition 3.2 (MCMC Sampling Principle)**.** Recall that a sufficiently nice Markov chain (MC) has an **invariant distribution** P_{inv}. Once the MC has converged to P_{inv}, each sample X_i from the chain has marginal distribution P_{inv}.

Markov chain Monte Carlo: Suppose we want to sample from a distribution with density p. If we can define a Markov chain with invariant distribution $P_{\text{inv}} \equiv p$ and sample $X_1, X_2, \ldots$ from the chain, then once it has converged,

$$X_i \sim p. \tag{19}$$

Remark 3.3 (Dependence in MCMC). For a Markov chain, X_{i+1} can depend on X_i. Hence, at least in principle, it is possible for an MCMC sampler to "remember" the previous step and remain in a high-probability location — unlike rejection sampling (Remark 3.1).

3.3 Continuous Markov Chains

Definition 3.4 (Continuous Markov Chain)**.** A continuous Markov chain is defined by an initial distribution P_{init} and a conditional probability $t(y \mid x)$, the **transition probability** or **transition kernel**. In the discrete case, $t(y = i \mid x = j)$ is the entry p_{ij} of the transition matrix.

Example 3.5 (Gaussian Random Walk)**.** A simple Markov chain on $\mathbb{R}^2$ can be defined by sampling

$$X_{i+1} \mid X_i = x_i \;\sim\; g(\,\cdot \mid x_i, \sigma^2),$$

where $g(x \mid \mu, \sigma^2)$ is a spherical Gaussian with fixed variance. The transition distribution is $t(x_{i+1} \mid x_i) := g(x_{i+1} \mid x_i, \sigma^2)$.

Definition 3.6 (Invariant Distribution — Continuous Case)**.** A distribution P_{inv} with density p_{inv} is **invariant** for a Markov chain with transition kernel t if

$$\int_{\mathcal{X}} t(y \mid x)\, p_{\text{inv}}(x)\, dx = p_{\text{inv}}(y). \tag{20}$$

This is the continuous analogue of the matrix equation $\boldsymbol{p} \cdot \boldsymbol{P}_{\text{inv}} = \boldsymbol{P}_{\text{inv}}$.

3.4 The Metropolis–Hastings Algorithm

3.4.1 Problems to Solve

Given a target density p, we need to:

1. Construct a Markov chain with invariant distribution p.
2. Handle the fact that we cannot start from $X_1 \sim p$ (if we could, sampling would be unnecessary).
3. Deal with the fact that the samples X_i are not i.i.d. (they are only marginally distributed as p).

3.4.2 Constructing the Markov Chain

Definition 3.7 (Metropolis–Hastings Kernel)**.** Given a continuous target distribution with density p:

1. Define a conditional probability $q(y \mid x)$ on $\mathcal{X}$ (the **proposal distribution**). This need not depend on p.
2. Define a **rejection kernel** (acceptance probability):

$$A(x_{i+1} \mid x_i) := \min\left\{1,\; \frac{q(x_i \mid x_{i+1})\, p(x_{i+1})}{q(x_{i+1} \mid x_i)\, p(x_i)}\right\}. \tag{21}$$

The normalization of p cancels in the quotient, so knowing $\tilde{p}$ suffices (Remark 1.5).

3. The transition probability of the chain is:

$$t(x_{i+1} \mid x_i) := q(x_{i+1} \mid x_i)\, A(x_{i+1} \mid x_i) + \delta_{x_i}(x_{i+1})\, c(x_i), \tag{22}$$

where $c(x_i) := \int q(y \mid x_i)\big(1 - A(y \mid x_i)\big)\, dy$ is the total probability that a proposal is generated and then rejected.

Algorithm 3.8 (Metropolis–Hastings Sampler)**.** At each step $i+1$, generate a proposal $X^* \sim q(\cdot \mid x_i)$ and $U_i \sim \text{Uniform}[0,1]$:

- If $U_i \leq A(X^* \mid x_i)$, **accept**: set $x_{i+1} := X^*$.
- If $U_i > A(X^* \mid x_i)$, **reject**: set $x_{i+1} := x_i$.

3.4.3 Stochastic Hill-Climbing Interpretation

Remark 3.9 (Hill-Climbing Behavior). From the definition of the MH acceptance probability (21), we certainly accept if

$$q(x_i \mid x_{i+1})\, p(x_{i+1}) > q(x_{i+1} \mid x_i)\, p(x_i).$$

This means:

- We always accept the proposal x_{i+1} if it increases the probability under p.
- If it decreases the probability, we still accept with a probability depending on the ratio.

The MH sampler resembles a gradient ascent algorithm on p that tends to move in the direction of increasing probability, but can also move "downhill" with a certain probability, so it does not get stuck at local maxima.

3.5 Burn-in and Convergence

3.5.1 Initial Distribution and Convergence

Theorem 3.10 (Fundamental Theorem on Markov Chains — Continuous Case)**.** *Suppose we sample $X_1 \sim P_{\text{init}}$ and $X_{i+1} \sim t(\cdot \mid x_i)$. This defines a distribution P_i of X_i, which can change from step to step. If the Markov chain is irreducible and aperiodic, then*

$$P_i \longrightarrow P_{\text{inv}} \qquad \text{as} \quad i \to \infty. \tag{23}$$

Remark 3.11 (Consequence for MCMC). If we can show that $P_{\text{inv}} \equiv p$, we do not have to know how to sample from p directly. Instead, we can start with *any* P_{init}, and will get arbitrarily close to p for sufficiently large i (Theorem 3.10).

3.5.2 Burn-in and Mixing Time

Definition 3.12 (Mixing Time and Burn-in)**.** The number m of steps required until $P_m \approx P_{\text{inv}} \equiv p$ is called the **mixing time** of the Markov chain. In MCMC samplers, the first m samples are called the **burn-in phase** and are discarded:

$$\underbrace{X_1, \ldots, X_{m-1}, X_m}_{\text{burn-in; discard}}, \qquad \underbrace{X_{m+1}, \ldots}_{\text{samples from (approx.) } p\text{; keep}}.$$

Remark 3.13 (Convergence Diagnostics). In practice, we do not know how large m is. There are a number of heuristic methods for assessing whether the sampler has mixed, often referred to as **convergence diagnostics**.

3.5.3 Sequential Dependence and Lag

Definition 3.14 (Lag)**.** Even after burn-in (Definition 3.12), the samples from a Markov chain are not i.i.d. The **lag** L is the number of steps needed for X_i and X_{i+L} to be approximately independent. After burn-in, we keep only every L-th sample and discard intermediate ones.

Definition 3.15 (Autocorrelation Function)**.** The most common method for estimating the lag (Definition 3.14) uses the **autocorrelation function**:

$$\mathrm{Auto}(X_i, X_j) := \frac{\mathbb{E}\big[(X_i - \mu_i)(X_j - \mu_j)\big]}{\sigma_i \, \sigma_j}, \tag{24}$$

where μ_i is the mean and σ_i the standard deviation of X_i. We compute $\mathrm{Auto}(X_i, X_{i+L})$ empirically from the sample for different values of L, and find the smallest L for which the autocorrelation is close to zero.

3.5.4 Convergence Diagnostics: Gelman–Rubin Criterion

Algorithm 3.16 (Gelman–Rubin Criterion)**.**

1. Start several chains at random initial points. For each chain k, the sample $X_i^{(k)}$ has a marginal distribution $P_i^{(k)}$.
2. The distributions $P_i^{(k)}$ will differ between chains in early stages.
3. Once the chains have converged, all $P_i = P_{\mathrm{inv}}$ are identical.
4. **Criterion:** Use a hypothesis test to compare $P_i^{(k)}$ for different k. Once the test does not reject anymore, assume that the chains are past burn-in (Definition 3.12).

3.5.5 Selecting a Proposal Distribution

Remark 3.17 (Proposal Variance Trade-off). Consider p as the target and q as a Gaussian proposal (Algorithm 3.8):

- If $\mathrm{Var}[q]$ is too large, the sampler will overstep p, leading to many rejections.
- If $\mathrm{Var}[q]$ is too small, many steps are needed for good coverage of the domain.

If p is unimodal and can be roughly approximated by a Gaussian, $\mathrm{Var}[q]$ should be chosen as the smallest covariance component of p. For complicated posteriors, choosing q with good performance requires prior knowledge about the posterior; mixture proposals (with one component for large steps and one for small steps) are one common strategy.

3.6 Summary: Metropolis–Hastings Sampler

Remark 3.18 (MH Sampler Summary). The key points of the Metropolis–Hastings algorithm (Algorithm 3.8) are:

- MCMC samplers construct a Markov chain with invariant distribution p (Definition 3.6).
- The MH kernel (Definition 3.7) is one generic way to construct such a chain from p and a proposal distribution q.
- Formally, q does not depend on p (but arbitrary choice usually means bad performance).
- We discard the first m samples as burn-in (Definition 3.12).
- After burn-in, we keep only every L-th sample (where L is the lag, Definition 3.14) to obtain approximately independent samples:

$$\underbrace{X_1, \ldots, X_m}_{\text{burn-in; discard}}, \underbrace{X_{m+1}, \ldots, X_{m+L-1}}_{\text{discard}}, \underbrace{X_{m+L}}_{\text{keep}}, \underbrace{X_{m+L+1}, \ldots}_{\text{discard}}, \underbrace{X_{m+2L}}_{\text{keep}}, \ldots$$

4 The Gibbs Sampler

4.1 Full Conditional Distributions

Definition 4.1 (Full Conditional Distribution)**.** Suppose $\mathcal{L}(X)$ is a distribution on $\mathbb{R}^D$, so $X = (X_1, \ldots, X_D)$. The conditional probability of the entry X_d given all other entries,

$$\mathcal{L}(X_d \mid X_1, \ldots, X_{d-1}, X_{d+1}, \ldots, X_D), \tag{25}$$

is called the **full conditional distribution** of X_d. If X has density p, we write $p(x_d \mid x_1, \ldots, x_{d-1}, x_{d+1}, \ldots, x_D)$.

4.2 The Gibbs Sampling Algorithm

Remark 4.2 (Why Gibbs Sampling). The Gibbs sampler is by far the most widely used MCMC algorithm. It is applicable whenever we can compute the full conditionals (Definition 4.1) for each dimension d, and it provides a generic way to derive a proposal distribution.

Algorithm 4.3 (Gibbs Sampler)**.** Suppose p is a density or mass function of a random vector with D entries. Given a random vector X_i, we generate X_{i+1} coordinate-by-coordinate as follows:

$$\begin{aligned}
X_{i+1,1} &\sim p(\cdot \mid X_{i,2}, \ldots, X_{i,D}) \\
&\vdots \\
X_{i+1,d} &\sim p(\cdot \mid X_{i+1,1}, \ldots, X_{i+1,d-1}, X_{i,d+1}, \ldots, X_{i,D}) \\
&\vdots \\
X_{i+1,D} &\sim p(\cdot \mid X_{i+1,1}, \ldots, X_{i+1,D-1})
\end{aligned} \tag{26}$$

Note: Each new draw $X_{i+1,d}$ is used immediately to update the next dimension $d+1$. The MCMC algorithm that generates vectors $X_1, X_2, \ldots$ as above is called a **Gibbs sampler**.

Proposition 4.4 (Gibbs as Metropolis–Hastings)**.** *For each dimension $d \in \{1, \ldots, D\}$, the Gibbs update (26) defines a process $X_{1,d}, X_{2,d}, \ldots$, which is itself a Markov process. One can show that this is a Metropolis–Hastings sampler (Algorithm 3.8). Hence, a Gibbs sampler with D full conditionals is a family of D coupled MH samplers. Moreover, these MH samplers all have acceptance probability* 1*: **proposals in Gibbs sampling are always accepted**.*

4.3 Application: Gibbs Sampling for MRFs

Example 4.5 (Full Conditionals of an MRF)**.** In a Markov random field (Definition 2.4) with D nodes, each dimension d corresponds to one vertex. The Markov property (12) implies that the full conditional (Definition 4.1) depends only on the neighbors. For instance, in a grid with 4-neighborhoods:

$$p(\theta_d \mid \theta_1, \ldots, \theta_{d-1}, \theta_{d+1}, \ldots, \theta_D) = p(\theta_d \mid \theta_{\text{left}}, \theta_{\text{right}}, \theta_{\text{up}}, \theta_{\text{down}}). \tag{27}$$

Example 4.6 (Gibbs Sampling for the Potts Model)**.** For the Potts model (Definition 2.6) with binary weights, the unnormalized full conditional is:

$$\begin{aligned}
&\tilde{p}(\theta_d \mid \theta_1, \ldots, \theta_{d-1}, \theta_{d+1}, \ldots, \theta_D) \\
&\qquad = \exp\big(\beta \left(\mathbb{I}\{\theta_d = \theta_{\text{left}}\} + \mathbb{I}\{\theta_d = \theta_{\text{right}}\} + \mathbb{I}\{\theta_d = \theta_{\text{up}}\} + \mathbb{I}\{\theta_d = \theta_{\text{down}}\}\right)\big).
\end{aligned} \tag{28}$$

This is clearly very efficiently computable, since we only need the unnormalized density for sampling (Remark 1.5).

Each step of the Gibbs sampler (Algorithm 4.3) generates a complete sample

$$\boldsymbol{\theta}_i = (\theta_{i,1}, \ldots, \theta_{i,D}),$$

where D is the number of vertices in the grid.

4.4 Burn-in Matters: A Cautionary Example

Example 4.7 (Insufficient Burn-in in MRF Sampling)**.** MRFs were introduced as tools for image smoothing and segmentation by D. and S. Geman in 1984. They sampled from a Potts model (Definition 2.6) with a Gibbs sampler (Algorithm 4.3), discarding 200 iterations as burn-in (Definition 3.12). The resulting samples exhibited "segmentation" patterns, leading computer vision researchers to conclude that MRFs are "natural" priors for image segmentation.

Sudderth and Jordan later ran a Gibbs sampler on the same model and recorded the state both after 200 iterations and again after 10,000 iterations. The key observations are:

- The "segmentation" patterns visible after 200 iterations are sampled from $P_{200} \neq P_{\text{inv}}$, not from the MRF distribution $p \equiv P_{\text{inv}}$.
- After 10,000 iterations, the patterns vanish: the true samples from the MRF look nearly uniform, confirming convergence.
- MRFs are **smoothness priors**, not segmentation priors. The earlier segmentation-like patterns were simply artifacts of insufficient burn-in.

Remark 4.8 (Lesson on Burn-in). Example 4.7 illustrates a fundamental pitfall of MCMC: drawing conclusions from samples generated before the chain has converged to its invariant distribution can be severely misleading. Adequate burn-in (Definition 3.12) and proper convergence diagnostics (Remark 3.13, Algorithm 3.16) are essential for reliable inference.